A Self-Addressing Radix Tree for On-Chain Limit Order Matching

Radix Matching Research Note

June 29, 2026

Abstract

This paper specifies a compact on-chain limit order book whose storage state is a binary radix tree over a 64-bit price-time key. Resting orders and internal aggregate nodes are represented by one 32-byte word: 24 bits of price, 192 bits of quantity, and 40 bits of decrementing nonce. The decrementing nonce gives earlier orders larger time-priority keys at a fixed price. Two conceptual books, bids and asks, coexist in one mapping from node words to child pairs. We formalize the node encoding, the bid and ask orderings, insertion, removal, matching, and claim/cancel semantics as a state transition system. We then compare the computational complexity of the radix construction with classical price-tree order books, off-chain relay systems, and automated market makers. The analysis distinguishes two variants. The fully aggregated variant described in the Warp and Hanji materials stores both subtree quantity and subtree value and can decide large fills in a number of steps bounded by the fixed key width. The strict one-word instantiation specified here stores quantity in the 192-bit aggregate field and therefore preserves compact storage while exact immediate quote settlement across mixed prices requires either leaf traversal or an additional value aggregate. This distinction is material to the complexity theorem and is made explicit.

1 Introduction

An open limit order book maintains two sets of resting orders. A bid order offers quote asset for base asset up to a limit price; an ask order offers base asset for quote asset down to a limit price. Price-time priority requires that more aggressive prices execute first and that orders at the same price execute in arrival order. Electronic open limit order books have a substantial market-structure literature; for example, Glosten analyzes equilibrium price schedules in an open limit order book [1].

The on-chain setting changes the data-structure problem. A conventional matching engine may scan a sequence of price levels and FIFO queues. This is acceptable in a trusted low-latency server, but it is problematic on a smart-contract platform where every storage read and write is paid by a transaction and every transaction is subject to a gas limit. Hybrid decentralized exchange systems such as 0x reduce on-chain work by relaying signed orders off-chain and settling matched orders on-chain [6]. Automated market makers avoid order matching entirely by trading against a pool according to a conservation function [9, 10]. These designs are important prior art, but they are not equivalent to a fully on-chain price-time-priority order book.

The radix design considered here follows the direction publicly presented by Poon and Jeffrey for Warp and subsequently described in Hanji documentation: store orders in radix trees keyed by price and order number, and store aggregate fields in internal nodes so matching can be bounded by key depth rather than by the number of filled orders [12, 13, 14]. The present paper gives a formal

specification for a strict EVM-oriented node layout in which each resting node is one `bytes32` word:

$$\underbrace{p}_{24\text{ bits}} \;\|\; \underbrace{q}_{192\text{ bits}} \;\|\; \underbrace{n}_{40\text{ bits}}.$$

The goal is not to replace a line-by-line audit. Rather, the goal is to state the mathematical objects precisely enough that implementation claims, invariant tests, and gas measurements can be evaluated against a common model, in the style of prior formal DeFi specifications such as the Runtime Verification treatment of the constant-product market maker [8].

Contributions. This note makes four contributions.

(i) It specifies a self-addressing one-word node encoding and side-dependent radix orderings for bids and asks.

(ii) It gives state-transition definitions for insertion, matching, cancel, and filled-order claim.

(iii) It proves fixed-depth bounds for insertion and removal and gives conditional bounds for fully aggregated matching.

(iv) It identifies the exact condition under which aggregate matching is logarithmic: the contract state must contain enough aggregate information to compute both the base quantity and quote value of a removed subtree.

2 Related Work

Classical limit order books. The standard engineering model for a high-performance order book stores price levels in a balanced search tree and stores orders at each price level in a FIFO list. Selph's widely circulated note describes this design and the usual operations of add, cancel, and execute [2]. This family of designs can provide efficient local updates in an in-memory engine, but a market order that crosses many price levels or many resting orders still exposes work proportional to the number of consumed orders or levels. In a smart contract, that linear work becomes a direct gas-boundedness problem.

Radix trees. Radix trees store keys by successive key fragments rather than by comparison with whole keys. Patricia tries and adaptive radix trees are standard examples of path-compressed radix structures [3, 4]. For fixed-width integer keys, the height of a binary radix tree is bounded by the key width. This property is well suited to an EVM design where a 64-bit key can be traversed in at most 64 branch decisions. Persistent-memory work also observes that the radix structure is determined by key prefixes and does not require rotations or rebalancing [5].

On-chain exchange alternatives. 0x uses off-chain order relay with on-chain settlement: signed orders are broadcast outside the blockchain and are submitted to a settlement contract by a counterparty when execution is desired [6]. DutchX used a Dutch-auction mechanism rather than a continuous order book [7]. AMM protocols replace bilateral order matching with pool-based trading governed by an invariant or conservation function; the AMM literature includes both formal specifications and systematized surveys [8, 9]. Uniswap v3 adds concentrated liquidity to the constant-product model, increasing capital efficiency relative to uniform liquidity over the entire price range [11]. These designs show that on-chain exchange is possible when the on-chain state transition is kept bounded, but they make different tradeoffs from a price-time-priority order book.

Warp and Hanji logarithmic matching. The Warp presentation by Poon and Jeffrey introduced an EVM-oriented logarithmic matching engine based on radix trees and aggregate internal nodes [12, 13]. Hanji's documentation describes the same high-level mechanism: separate bid and ask radix trees, a price-plus-order-number key, internal nodes containing subtree sums, and a 64-bit key giving a maximum height of 64 and an execution bound of up to 127 steps [14]. The present work formalizes a closely related self-addressing storage layout with a stricter one-word node representation.

3 Notation and Encoding

Let

$$P = \{1, \dots, 2^{24} - 1\}, \qquad Q = \{1, \dots, 2^{192} - 1\}, \qquad N = \{0, \dots, 2^{40} - 1\}.$$

The value $p = 0$ is excluded for valid external orders. The nonce is assigned by the contract and decreases from $2^{40} - 1$. Thus, at a fixed price, an earlier order has a larger nonce.

Definition 1 (Order word)**.** *For $p \in P$, $q \in Q$, and $n \in N$, define*

$$W(p, q, n) = p2^{232} + q2^{40} + n.$$

The corresponding field extractors are

$$\text{price}(W) = p, \qquad \text{qty}(W) = q, \qquad \text{nonce}(W) = n.$$

Definition 2 (Path and sort keys)**.** *The raw path key of a node is*

$$\kappa(W) = \text{price}(W)2^{40} + \text{nonce}(W).$$

The side-dependent sort keys are

$$\kappa_B(W) = \text{price}(W)2^{40} + \text{nonce}(W)$$

for bids and

$$\kappa_A(W) = (2^{24} - 1 - \text{price}(W))2^{40} + \text{nonce}(W)$$

for asks.

The bid key orders larger prices to the right. The ask key reverses price so that lower ask prices also move to the right. In both books, larger nonce means earlier time priority and therefore higher rank at a fixed price. The result is that the best executable liquidity is found by traversing the right side of either tree.

Definition 3 (Side key)**.** *The zero-quantity side key associated with an order is*

$$\sigma(W(p, q, n)) = W(p, 0, n).$$

It is used as a compact side marker for filled resting orders. It is not a valid external order because external orders require nonzero quantity.

4 Radix Tree Model

Each book is a binary Patricia-style radix tree over a 64-bit sort key. Leaves are order words. Internal nodes are also represented by one 32-byte word and are used as storage addresses into a single mapping

$$\mathsf{tree} : \{0,1\}^{256} \to \{(\mathsf{leftNode}, \mathsf{rightNode})\}.$$

The two conceptual trees coexist in this mapping because their roots are separate variables and reachability is defined from the corresponding root.

Definition 4 (Common prefix and branch depth)**.** *For 64-bit keys x, y, let*

$$\mathrm{lcp}(x, y) = \min\{d \in \{0, \ldots, 63\} : x_d \neq y_d\},$$

where bit 0 is the most significant bit. If $x = y$, define $\mathrm{lcp}(x, y) = 64$. A valid branch has children with distinct keys, so its branch depth is $d < 64$.

Definition 5 (Branch ordering)**.** *For side $s \in \{A, B\}$ and branch children L, R, let*

$$d = \mathrm{lcp}(\kappa_s(L), \kappa_s(R)).$$

The child with bit 0 at depth d is the left child, and the child with bit 1 at depth d is the right child.

Definition 6 (One-word branch address)**.** *For children L, R, define the branch quantity and raw boundary key by*

$$Q(L, R) = \mathrm{qty}(L) + \mathrm{qty}(R), \qquad b(L, R) = \max\{\kappa(L), \kappa(R)\}.$$

The branch address is

$$\mathsf{br}(L, R) = W\left(\left\lfloor b(L, R)/2^{40} \right\rfloor, Q(L, R), b(L, R) \bmod 2^{40}\right).$$

This address is self-describing in the same bit layout as an order. The price and nonce fields encode the raw boundary key; the quantity field encodes the aggregate base quantity of the two child subtrees. No additional branch nonce, tag, or namespace is introduced.

Invariant 1 (Well-formed branch)**.** *A reachable branch node X is well formed if*

$$X = \mathsf{br}(L, R), \qquad \mathsf{tree}[X] = (L, R),$$

and L, R are nonzero reachable nodes satisfying the branch ordering for the side of the root from which X is reached.

Invariant 2 (Aggregate quantity)**.** *Let $\mathcal{L}(X)$ be the multiset of leaves below a reachable node X. The quantity field of X satisfies*

$$\mathrm{qty}(X) = \sum_{Y \in \mathcal{L}(X)} \mathrm{qty}(Y).$$

Remark 1. *The self-addressing construction moves structural information into the node word itself. Correctness therefore depends on the reachable-tree invariants, not on a separate branch identifier. Implementations should test that every reachable branch is two-child, ordered by the appropriate side key, and has a quantity equal to the sum of reachable leaves.*

5 State Transition System

The exchange state is

$$S = (T_B, T_A, O, \nu, C_B, C_Q),$$

where T_B and T_A are the bid and ask roots, O is the owner-of-order mapping, ν is the next nonce, and C_B, C_Q are contract balances of base and quote tokens. Token balances of external accounts are omitted except when required for conservation statements.

5.1 Insertion

An external order must have nonce field zero. If it rests, the contract assigns nonce ν, then decrements ν.

Definition 7 (Resting bid)**.** *For $p \in P$, $r \in Q$, and $\nu > 0$, define*

$$X = W(p, r, \nu).$$

The resting bid transition inserts X into T_B according to key $\kappa_B(X)$, sets $O[X]$ to the trader, and sets $\nu' = \nu - 1$.

Definition 8 (Resting ask)**.** *The resting ask transition is identical except that X is inserted into T_A according to $\kappa_A(X)$.*

Lemma 1 (Insertion depth)**.** *Let $h = 64$. Inserting a new leaf into a well-formed binary radix tree over 64-bit keys touches at most h branch depths before either finding an empty root, splitting a leaf, or descending into one child.*

Proof. At each non-leaf step, the algorithm compares the new key with the children's common-prefix depth and either creates a new branch at the first earlier disagreement or descends to the child selected by the next key bit. The depth strictly increases along a descent and cannot exceed 63 for distinct 64-bit keys. Therefore there are at most 64 branch decisions. □

5.2 Matching Semantics

Let rank_B order bids by descending price and descending nonce, and let rank_A order asks by ascending price and descending nonce. In either book, the radix rightmost order has the best rank under its side ordering.

Definition 9 (Crossing predicate)**.** *A bid with limit price p crosses an ask leaf Y if*

$$\text{price}(Y) \leq p.$$

An ask with limit price p crosses a bid leaf Y if

$$\text{price}(Y) \geq p.$$

Definition 10 (Leaf fill)**.** *Let an incoming order have remaining base quantity r, and let Y be a crossing resting leaf. The fill amount is*

$$\delta = \min\{r, \text{qty}(Y)\}.$$

The remaining incoming quantity becomes $r - \delta$. The resting leaf is removed if $\delta = \mathrm{qty}(Y)$ and is replaced by

$$W(\mathrm{price}(Y), \mathrm{qty}(Y) - \delta, \mathrm{nonce}(Y))$$

otherwise. The quote value of the fill is

$$\mathrm{val}(Y, \delta) = \mathrm{price}(Y)\delta.$$

Definition 11 (Linear reference semantics)**.** *For a bid taker, list ask leaves in increasing price and decreasing nonce at equal price:*

$$A_1, A_2, \ldots, A_m.$$

The reference match consumes the longest prefix, possibly followed by one partial leaf, such that every consumed leaf crosses the bid limit and the total consumed quantity does not exceed the incoming quantity. Ask takers are defined symmetrically over bid leaves in decreasing price and decreasing nonce at equal price.

The radix implementation is correct if its right-first traversal produces the same consumed sequence as the linear reference semantics.

Theorem 1 (Price-time priority)**.** *For a well-formed bid or ask tree, right-first traversal by the side-dependent sort key visits crossing leaves in the same price-time order as the linear reference semantics.*

Proof. For bids, the sort key is price 2^{40}+nonce. Thus larger price sorts right of smaller price, and at equal price larger nonce sorts right of smaller nonce. For asks, the sort key replaces price with $2^{24} - 1 -$price, so smaller price sorts right of larger price, while the nonce ordering is unchanged. A binary radix tree preserves sorted order under in-order traversal; therefore right-first traversal produces descending sort-key order, which is exactly price-time priority for the corresponding side. □

5.3 Cancel and Claim

Cancel is also the filled-order claim operation. The owner mapping is indexed by the original resting order word, not by the reduced leaf that may remain after a partial fill.

Definition 12 (Cancel search)**.** *Given an order X, cancellation first searches T_B by $\kappa_B(X)$. If no leaf is removed, it searches T_A by $\kappa_A(X)$. If neither search removes a leaf, the transition checks the side marker $O[\sigma(X)]$ to determine whether a fully filled claim exists.*

Definition 13 (Cancel payout)**.** *Let $q_0 = \mathrm{qty}(X)$ be the original quantity and let q_r be the remaining quantity found in the book, or zero if the order is fully filled. Let $q_f = q_0 - q_r$. For a bid owner, the payout is*

$$(B, Q) = (q_f, \mathrm{price}(X)q_r).$$

For an ask owner, the payout is

$$(B, Q) = (q_r, \mathrm{price}(X)q_f).$$

After payout, $O[X]$ is deleted; if the order was fully filled, the side marker $O[\sigma(X)]$ is also deleted.

This claim model makes fill execution and maker settlement asynchronous. At match time, the taker receives the asset owed by the matched resting orders. Makers later call cancel to claim filled proceeds and any unfilled remainder.

6 Complexity Analysis

Let h be the key width. In this design $h = 64$. Let z be the number of resting leaves whose quantities are actually examined during a match.

Theorem 2 (Insertion and cancel bound)**.** *For a well-formed tree over 64-bit keys, insertion and single-order removal each require $O(h)$ radix decisions. Since $h = 64$, this is a constant upper bound with respect to the number of orders in the book.*

Proof. Insertion follows Lemma 1. Removal follows the same unique path selected by successive key bits. If the target key diverges before a branch depth, the key is absent and the search terminates. Otherwise the search descends at most one child per depth until a leaf is reached or absence is proven. The depth is bounded by 64. □

6.1 Fully Aggregated Matching

The asymptotically strongest version of the Warp/Hanji construction stores two aggregate fields in each internal node:

$$Q(X) = \sum_{Y \in \mathcal{L}(X)} \mathrm{qty}(Y), \qquad V(X) = \sum_{Y \in \mathcal{L}(X)} \mathrm{price}(Y)\,\mathrm{qty}(Y).$$

Then a matching algorithm can remove an entire crossing subtree X when $Q(X) \le r$, immediately adding $V(X)$ to quote settlement.

Theorem 3 (Aggregated fill bound)**.** *Assume every branch stores both $Q(X)$ and $V(X)$, and assume execution of removed makers is represented asynchronously by claim state rather than by per-maker transfers in the taker transaction. A marketable incoming order can be matched by removing maximal covered subtrees and at most one partial boundary leaf in $O(h)$ branch decisions. For a binary tree, a right-spine search followed by at most one complementary left-spine search visits at most $2h - 1$ decision points.*

Proof. At each depth on the best-side spine, the algorithm compares the incoming remaining quantity with the aggregate quantity of a candidate subtree. If the candidate subtree is fully covered, the subtree is removed as one unit and its aggregate value is added to settlement. If it is not covered, the algorithm descends into that subtree. Each descent increases depth. A binary trie over h bits has height at most h. The search may traverse one spine to find the first non-covered subtree and one adjacent spine to repair the boundary, hence at most $2h - 1$ decision points. No step depends on the number of leaves contained in a fully removed subtree. □

This theorem is the source of the superior computational complexity claim: the matching decision is bounded by the key width instead of the number of filled orders or crossed price points. It is the same class of claim described in the Hanji documentation for a 64-bit key and in the Warp presentation materials [14, 12].

6.2 Strict One-Word Quantity-Only Instantiation

The strict node layout in this paper allocates the 192-bit middle field to quantity. It does not also store a quote-value aggregate for mixed price subtrees. Therefore, for exact immediate quote settlement across a mixed-price subtree, one of the following must hold:

(a) the subtree is known to contain a single price, so $V(X) = \mathrm{price}(X)Q(X)$;

(b) an additional value aggregate is present in state or derivable from another commitment; or

(c) the matcher traverses leaves or smaller subtrees until the quote value is exactly computed.

Corollary 1 (Quantity-only exact settlement)**.** *For the strict one-word layout without an additional value aggregate, exact immediate quote settlement has complexity $O(h+z)$ in the number of visited matching leaves in the usual trie traversal model. In the worst case, z may be the number of filled leaves. Insertion, single-order cancel, and membership proofs remain $O(h)$.*

Proof. The branch quantity determines whether enough base quantity exists below a subtree, but it does not determine the sum $\sum_i \mathrm{price}(Y_i)\,\mathrm{qty}(Y_i)$ when the subtree contains multiple prices. Exact quote transfer must therefore obtain price and quantity data from each price component contributing to the fill, unless an equivalent value aggregate is available. Each visited leaf lies below a path of length at most h, and trie traversal visits a bounded number of branch nodes per visited leaf plus boundary nodes. □

Remark 2. *This corollary is not a defect in the mathematical model; it is a storage-design tradeoff. A one-word node minimizes state per reachable node. A double-sum node gives the stronger fill bound but requires either a wider node, a second aggregate commitment, or a different encoding of price, quantity, and value.*

7 Collateral and Accounting

The design is non-custodial in the limited sense that user balances move through ERC-20 transfers controlled by the contract state transition. The contract does not assume an off-chain matcher has custody. It does, however, rely on configured tokens behaving as exact ERC-20 assets. Fee-on-transfer, rebasing, mint-on-transfer, or adversarially inconsistent tokens are outside the exact-accounting model.

Definition 14 (Bid collateral)**.** *When a bid with price p and quantity q rests, the contract must hold pq quote units for its unfilled remainder. If a bid partially matches and rests r units, quote collateral for the resting remainder is pr, while quote paid for immediate fills is computed from the prices of matched asks.*

Definition 15 (Ask collateral)**.** *When an ask with quantity q is submitted, the contract must receive q base units. Any immediate fills produce quote proceeds for the ask taker; any unfilled remainder stays as base collateral in the book.*

Theorem 4 (Conservation under exact tokens)**.** *Assume all ERC-20 transfers debit the source and credit the recipient by exactly the requested amount, and assume every successful state transition follows the payout equations above. Then contract balances equal the sum of unfilled resting collateral plus unclaimed filled proceeds.*

Proof. The statement follows by induction over transitions. Insertion adds exactly the collateral associated with the new resting remainder. Matching transfers taker proceeds immediately and records maker claim state by retaining the corresponding collateral in the contract. Cancel or claim deletes the owner entry and transfers exactly the amount computed from the original order and remaining quantity. Each transition therefore preserves the equality between contract balances and outstanding obligations. □

8 Implementation Considerations

8.1 One mapping for two books

The storage type is

$$\mathsf{tree}[X] = (L, R),$$

where X, L, R are node words. There is no separate bid-tree mapping and ask-tree mapping. Coexistence is achieved by separate roots T_B, T_A and by side-dependent traversal keys. Any security argument must therefore be stated over reachability from roots: a storage entry is active for a book only if it is reachable from that book's root by valid side routing.

8.2 Decrementing nonce

Let $\nu_0 = 2^{40} - 1$. If m orders have rested, and no nonce exhaustion has occurred, then

$$\nu = \nu_0 - m.$$

Since earlier orders have larger nonce values, appending the nonce to the price key gives FIFO priority at each price without a linked list. The nonce is part of the order identity and of the radix path.

8.3 Partial fills

When a leaf is partially filled, the remaining leaf has the same price and nonce but a smaller quantity. The owner mapping remains keyed by the original order. Consequently, an implementation must reject attempts to claim using the reduced leaf word as though it were the original order. The side key $\sigma(X)$ exists only for fully filled orders and is zero-quantity by construction.

8.4 Reentrancy and external calls

Because token transfers are external calls, a production implementation requires a reentrancy guard around fill and cancel. The formal model above treats each state transition as atomic. The implementation must restore this atomicity in the EVM execution model by preventing callbacks from observing or mutating intermediate state.

9 Comparison

10 Conclusion

A binary radix tree over the 64-bit price-time key provides a natural fixed-depth order-book representation for an EVM setting. The decrementing nonce removes the need for per-price FIFO lists: price and time priority are both embedded in the key. The self-addressing `bytes32` layout further reduces storage by representing orders and aggregate branches in the same word format and by using one mapping for both conceptual books.

The main complexity result is conditional and precise. If internal nodes carry enough aggregate information to remove a covered subtree and settle its value, then matching work is bounded by the fixed key depth, independent of the number of resting orders represented by the removed subtree. This is the logarithmic matching-engine idea described in the Warp and Hanji references. If the implementation stores only aggregate quantity in the one-word node, it retains fixed-depth

Design	On-chain matching work	Principal tradeoff
Price tree plus FIFO lists	Linear in filled orders or crossed price levels for large marketable orders	Efficient in-memory design, but the filled range is unbounded on-chain
Off-chain relay with on-chain settlement	On-chain work is settlement of submitted signed orders	Matching and order availability are performed outside contract state
AMM / CFMM	Constant or small bounded pool update	No price-time-priority book; price is determined by a conservation function
Double-sum radix LME	$O(h)$, with $h = 64$, for aggregate matching decisions	Requires aggregate quantity and aggregate value in branch state
Strict one-word quantity radix	$O(h)$ insert/cancel; exact mixed-price fill requires visited value computation	Minimal node footprint; full logarithmic mixed-price settlement requires an additional value aggregate or deferred execution representation

Table 1: Computational comparison of exchange-state designs.

insertion and cancellation and obtains a compact order-book representation, but exact immediate quote settlement across mixed prices must recover value by leaf traversal or by another aggregate mechanism. A complete production specification should therefore state which aggregate fields are present and which execution semantics are immediate versus asynchronous.

References

[1] Lawrence R. Glosten. Is the electronic open limit order book inevitable? *The Journal of Finance*, 49(4):1127–1161, 1994. https://doi.org/10.1111/j.1540-6261.1994.tb02450.x.

[2] W. K. Selph. How to build a fast limit order book. Blog post, archived 2011. https://web.archive.org/web/20110219163448/http://howtohft.wordpress.com/2011/02/15/how-to-build-a-fast-limit-order-book/.

[3] Donald R. Morrison. PATRICIA – Practical algorithm to retrieve information coded in alphanumeric. *Journal of the ACM*, 15(4):514–534, 1968. https://doi.org/10.1145/321479.321481.

[4] Viktor Leis, Alfons Kemper, and Thomas Neumann. The adaptive radix tree: ARTful indexing for main-memory databases. In *IEEE ICDE*, 2013. https://www.db.in.tum.de/~leis/papers/ART.pdf.

[5] Se Kwon Lee, K. Hyun Lim, Hyunsub Song, Beomseok Nam, and Sam H. Noh. WORT: Write optimal radix tree for persistent memory storage systems. In *15th USENIX Conference on File and Storage Technologies*, 2017. https://www.usenix.org/conference/fast17/technical-sessions/presentation/lee-se-kwon.

[6] 0x Project. 0x protocol specification v2. https://github.com/0xProject/0x-protocol-specification/blob/master/v2/v2-specification.md.

[7] Gnosis. DutchX documentation. https://dutchx.readthedocs.io/.

[8] Yi Zhang, Xiaohong Chen, and Daejun Park. Formal specification of constant product ($x \times y = k$) market maker model and implementation. Runtime Verification, 2018. https://github.com/runtimeverification/verified-smart-contracts/blob/master/uniswap/x-y-k.pdf.

[9] Jiahua Xu, Krzysztof Paruch, Simon Cousaert, and Yebo Feng. SoK: Decentralized exchanges (DEX) with automated market maker (AMM) protocols. *ACM Computing Surveys*, 55(11), 2023. https://arxiv.org/abs/2103.12732.

[10] Guillermo Angeris, Alex Evans, and Tarun Chitra. When does the tail wag the dog? Curvature and market making. arXiv:2012.08040, 2020. https://arxiv.org/abs/2012.08040.

[11] Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. Uniswap v3 core. Whitepaper, 2021. https://app.uniswap.org/whitepaper-v3.pdf.

[12] Joseph Poon and Christopher Jeffrey. Warp – efficient decentralized exchange. EthCC 2023 presentation video, timestamp supplied by the project. https://www.youtube.com/live/E4YkppHQZgw?si=GunMcT_DjjNiRIW4&t=5855.

[13] EthCC 2023 program data. Joseph Poon and Christopher Jeffrey, Warp – Efficient Decentralized Exchange. https://gist.github.com/valer-cara/cf41cf56a6184dbfa8823350ee24a38a.

[14] Hanji. Matching engine architecture documentation. https://docs.hanji.io/architecture/matching-engine.